

STM32 USART Bare-Metal Driver (Register Level)

Overview

This repository contains a **bare-metal USART (Universal Synchronous/Asynchronous Receiver Transmitter) driver** written in **embedded C** for STM32 micro-controllers.

The driver is implemented using **direct register access**, without relying on STM32 HAL or CMSIS driver libraries.

The goal of this driver is to:

- Understand USART peripheral internals
- Practice low-level driver development
- Provide a reusable and configurable USART abstraction

This driver supports:

- Polling-based transmission and reception
 - Interrupt-based transmission and reception
 - Configurable baud rate, word length, parity, stop bits, and hardware flow control
-

Target Hardware

- **MCU Family:** STM32 (tested/planned for STM32F1 class devices)
- **USART Instances:**
 - USART1
 - USART2
 - USART3
 - UART4
 - UART5

 Exact peripheral availability depends on the MCU variant.

Driver Architecture

Files

- `usart_driver.c` → Driver implementation
- `usart_driver.h` → Register definitions, macros, configuration structures

Design Pattern

- Handle-based design (USART_Handle_t)
 - Configuration via structure (USART_Config_t)
 - Separation of:
 - Clock control
 - Peripheral configuration
 - Data transfer
 - Interrupt handling
-

USART Handle Structure

The driver uses a handle structure (USART_Handle_t) to store:

- USART peripheral base address
- Configuration parameters
- TX/RX buffer pointers
- Length and busy state information

This allows multiple USART instances to be managed independently.

Initialization Flow

Typical USART initialization sequence:

1. Enable peripheral clock
 2. Configure mode (TX / RX / TXRX)
 3. Configure word length
 4. Configure parity control
 5. Configure stop bits
 6. Configure hardware flow control
 7. Configure baud rate
 8. Enable USART peripheral
-

API Reference

1. Peripheral Clock Control

```
void USART_PerioClockControl(USART_RegDef_t *pUSARTx, uint8_t ENorDI);
```

Description

Enables or disables the USART peripheral clock using RCC registers.

Parameters

- `pUSARTx` → USART base address
 - `ENorDI` → `ENABLE` or `DISABLE`
-

2. Peripheral Enable / Disable

```
void USART_PeripheralControl(USART_RegDef_t *pUSARTx, uint8_t EnorDi);
```

Description

Controls the UE (USART Enable) bit in `CR1`.

3. USART Initialization

```
void USART_Init(USART_Handle_t *pUSARTHandle);
```

Description

Configures:

- Mode (TX / RX / TXRX)
 - Word length (8-bit / 9-bit)
 - Parity (None / Even / Odd)
 - Stop bits
 - Hardware flow control
 - Baud rate configuration
-

4. Baud Rate Configuration

```
void USART_SetBaudRate(USART_RegDef_t *pUSARTx, uint32_t BaudRate);
```

Description

- Calculates `USARTDIV` using APB clock frequency
- Programs the `BRR` register
- Supports APB1 and APB2 based USARTs

Formula Used

$$\text{USARTDIV} = \text{PCLKx} / (16 \times \text{BaudRate})$$

(Implemented using integer math to avoid floating point)

5. Blocking Data Transmission

```
void USART_SendData(USART_Handle_t *pUSARTHandle, uint8_t *pTxBuffer,
uint32_t Len);
```

Description

- Polls TXE flag
 - Writes data to DR register
 - Supports:
 - 8-bit / 9-bit word length
 - With or without parity
 - Waits for TC flag at the end
-

6. Blocking Data Reception

```
void USART_ReceiveData(USART_Handle_t *pUSARTHandle, uint8_t *pRxBuffer,
uint32_t Len);
```

Description

- Polls RXNE flag
 - Reads data from DR register
 - Correctly masks data based on:
 - Word length
 - Parity configuration
-

7. Interrupt-Based Transmission

```
uint8_t USART_SendDataIT(USART_Handle_t *pUSARTHandle, uint8_t
*pTxBuffer, uint32_t Len);
```

Description

- Initializes TX state
- Enables:
 - TXE interrupt
 - TC interrupt
- Actual data transfer handled inside ISR

Return Value

- Current transmission state
-

8. Interrupt-Based Reception

```
uint8_t USART_ReceiveDataIT(USART_Handle_t *pUSARTHandle, uint8_t
*pRxBuffer, uint32_t Len);
```

Description

- Initializes RX state
 - Enables RXNE interrupt
 - Data reception handled inside ISR
-

9. Flag Status API

```
uint8_t USART_GetFlagStatus(USART_RegDef_t *pUSARTx, uint8_t
StatusFlagName);
```

Description

- Reads status flags from USART_SR
 - Used internally by polling APIs
-

10. Flag Clear API

```
void USART_ClearFlag(USART_RegDef_t *pUSARTx, uint16_t StatusFlagName);
```

Description

- Clears specific status flags
 - Useful for error and event handling
-

Supported Features

- ✓ Polling-based TX/RX
 - ✓ Interrupt-based TX/RX
 - ✓ Configurable baud rate
 - ✓ Hardware flow control (RTS / CTS)
 - ✓ 8-bit and 9-bit data
 - ✓ Parity support
-

Coding Style & Philosophy

- No HAL or LL drivers
 - No magic numbers
 - Explicit bit manipulation
 - Close adherence to STM32 reference manual
-